

Лабораторна робота №5

Навчання без вчителя

Мета роботи: на основі розгляду прикладів з використанням програмного коду на Python, вивчити найпопулярніші методи навчання без вчителя для кластеризації даних: метод k-середніх (k-means); метод ієрархічної кластеризації; метод t-SNE, метод кластеризації на основі щільності DBSCAN. Самостійно вивчити програмні функції бібліотек Python, які використовуються в даних прикладах. Виконати завдання, які надано після кожного розділу роботи.

Термінологія:

- **Feature** (Особливості): вхідна змінна, яка використовується для створення прогнозів.
- **Predictions** (Прогнози): вихідні дані моделі за наявності вхідного прикладу.
- **Example** (Приклад): Набір даних. Приклад, який зазвичай містить один чи кілька об'єктів.
- **Label** (Мітки): результат функції.

1. Підготовка вибірки для кластеризації даних

Для створення прогнозів будемо використовувати класичний [набір даних ірисів Фішера](#). Датасет представляє набір із 150 записів з п'ятьма атрибутами в наступному порядку: довжина чашолистка (sepal length), ширина чашолистка (sepal width), довжина пелюстки (petal length), ширина пелюстки (petal width) та клас, що відповідає одному з трьох видів: Iris Setosa, Iris Versicolor або Iris Virginica, позначених відповідно 0, 1, 2. Алгоритм повинен аналізувати чотири властивості однієї конкретної квітки та передбачати, до якого класу (виду ірису) він належить. Мітки, що є в наборі даних, можна використовувати для оцінки якості передбачення.

Для вирішення задач кластеризації даних будемо використовувати Python, бібліотеку **scikit-learn** (<https://neurohive.io/ru/osnovy-data-science/vvedenie-v-scikit-learn/>) - для завантаження та обробки набору даних, бібліотеку **matplotlib** (<https://pythonworld.ru/novosti-mira-python/scientific-graphics-in-python.html>) - для візуалізації.

Нижче представлено програмний код для дослідження початкового набору даних.

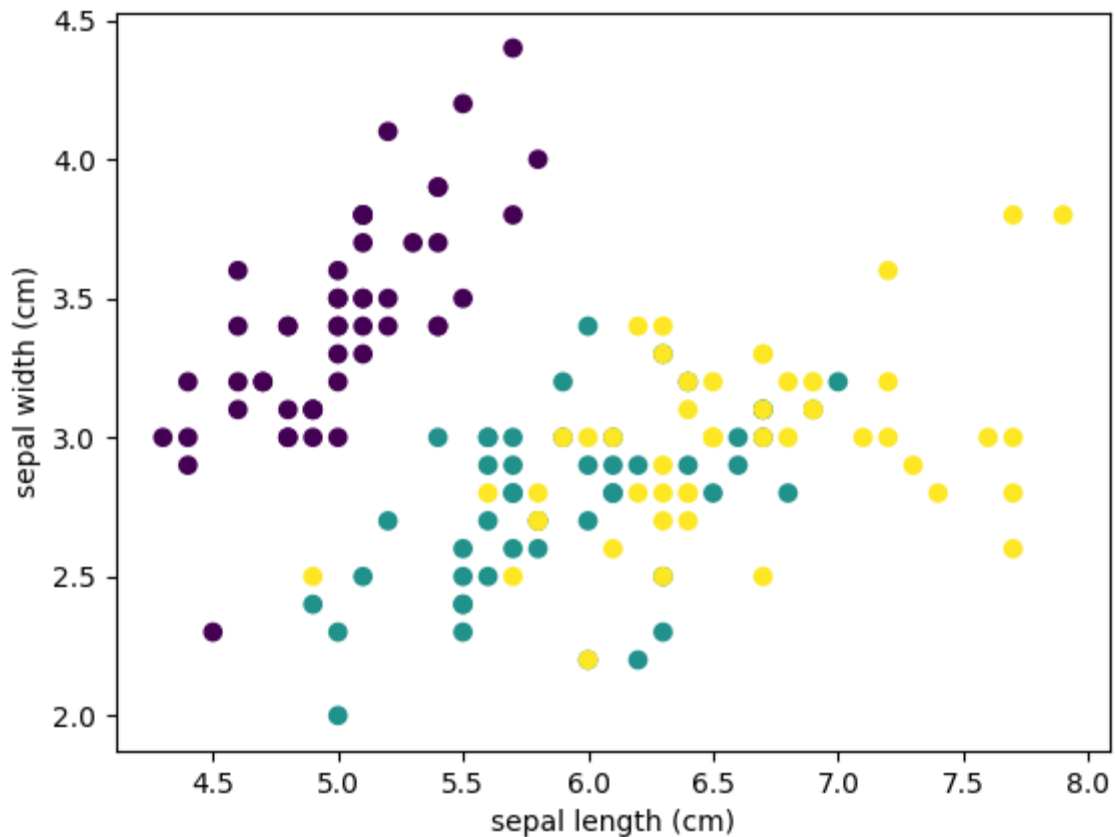


Рис.1. Візуалізація початкового розподілу прикладів у двовимірному просторі ознак

На діаграмі фіолетовим кольором позначений вид Setosa, зеленим – Versicolor та жовтим – Virginica. При побудові було взято *лише дві ознаки*.

Завдання 1: Проаналізуйте, як розподіляються види ірисів при інших комбінаціях параметрів (ознак).

2. Метод k-середніх

Найбільш популярним алгоритмом кластеризації даних є [метод k-середніх](#). Це ітеративний алгоритм кластеризації, що ґрунтується на мінімізації сумарних квадратичних відхилень точок кластерів від центроїдів (середніх координат) цих кластерів.

Спочатку вибирається бажана кількість кластерів. Оскільки нам відомо, що в нашому наборі даних є 3 класи, встановимо параметр моделі **n_clusters**, що дорівнює трьом.

Далі, випадковим чином із вхідних даних вибираються три елементи вибірки, яким у відповідність ставляться три кластери. Тепер у кожен із кластерів буде включено по одній точці, кожна з яких буде центроїдом цього кластера.

Далі шукаємо найближчого сусіда, поточного центроїду. Додаємо точку до відповідного кластера та перераховуємо положення центроїду з урахуванням координат нових точок. Алгоритм закінчує роботу, коли координати кожного центроїду перестають змінюватись. Центроїд кожного кластера в результаті є набором значень ознак, що описують усереднені параметри виділених класів. Код алгоритму матиме вигляд:

```
# Імпортуємо бібліотеки
from sklearn import datasets
from sklearn.cluster import KMeans

# Завантажуємо набір даних
iris_df = datasets.load_iris()

# Описуємо модель
model = KMeans(n_clusters=3)

# Проводимо моделювання
model.fit(iris_df.data)

# Передбачення на одиничному прикладі
predicted_label = model.predict([[7.2, 3.5, 0.8, 1.6]])

# Передбачення на всьому наборі даних
all_predictions = model.predict(iris_df.data)

# Виводимо результати передбачення
print(predicted_label)
print(all_predictions)
```

Результат:

[illegible]

При виведенні даних слід розуміти, що алгоритм нічого не знає про нумерацію класів, і числа 0, 1, 2 – це лише номери кластерів, визначених у результаті роботи алгоритму. Оскільки вихідні точки вибираються випадковим чином, висновок

буде змінюватися від одного запуску до іншого. Характерною особливістю набору даних ірисів Фішера є те, що один клас (Setosa) легко відокремлюється від двох інших. Це помітно й у наведеному прикладі.

Завдання 2: Візуалізуйте отримані результати кластеризації.

3. Ієрархічна кластеризація

Ієрархічна кластеризація, як випливає з назви, є алгоритм, який будує ієрархію кластерів. Цей алгоритм починає роботу з того, що кожному екземпляру даних зіставляється власний кластер. Потім два найближчих кластери об'єднуються в один і так далі, доки не буде утворено один загальний кластер.

Результат ієрархічної кластеризації може бути представлений за допомогою **дендрограми** (рис.2). Розглянемо цей тип кластеризації на прикладі даних різних видів зерна (використовується бібліотека **Pandas**).

```
# Імпортуємо бібліотеки
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt
import pandas as pd

# Створюємо датафрейм
seeds_df = pd.read_csv(
    "http://qps.ru/jNZUT")

# Виключаємо інформацію про зразки зерна, зберігаємо її для подальшого
використання
varieties = list(seeds_df.pop('grain_variety'))

# Вилучаємо виміри як масив NumPy
samples = seeds_df.values

# Реалізація ієрархічної кластеризації за допомогою функції linkage
mergings = linkage(samples, method='complete')

# Будуємо дендрограму, вказавши параметри, що зручні для відображення
dendrogram(mergings,
            labels=varieties,
            leaf_rotation=90,
            leaf_font_size=6,
            )

plt.show()
```

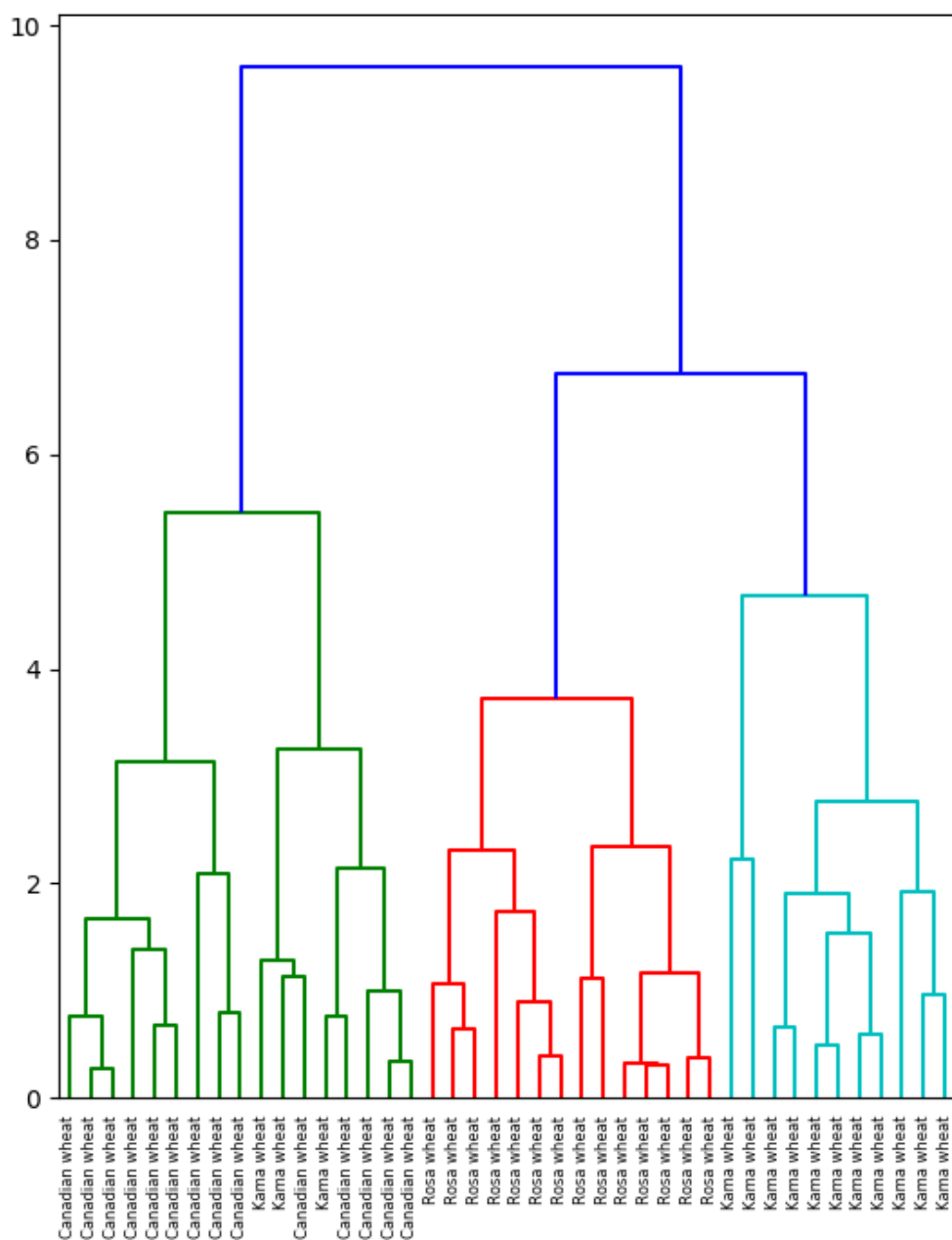


Рис.2. Дендрограма результатів ієрархічної кластеризації

Дендрограма наочно демонструє, що в результаті ієрархічної кластеризації даних природно сталося їхнє розбиття на три кластери, позначені на малюнку різним кольором. При цьому початкове число кластерів не задавалося.

Завдання 3: Здійсніть ієрархічну кластеризацію для датасету "Іриси Фішера". Побудуйте дендрограму. Порівняйте результати кластеризації двох методів для одного набору даних. Зробіть висновки: який метод краще підходить для великих наборів даних з точки зору часових та обчислювальних витрат і чому; який метод точніший; який метод більш чутливий до "зашумлення" даних.

4. Зниження розмірності з використанням методу t-SNE

Метод t-SNE (англ. t-distributed stochastic neighbor embedding - розподілене стохастичне сусіднє вкладення) є одним із методів навчання без вчителя, що використовуються для візуалізації, наприклад, відображення простору високої розмірності в дво- або тривимірний простір.

Метод відображає кожен об'єкт простору високої розмірності у дво- або трикоординатну точку таким чином, що близькі за характеристиками елементи даних у багатовимірному просторі (наприклад, датасеті з великим числом стовпців) проектуються в сусідні точки, а різномірні об'єкти з більшою ймовірністю моделюються точками, далеко віддаленими один від одного. Математичний опис роботи методу можна знайти [тут](#).

Візьмемо датасет "Іриси Фішера" та подивимося, як здійснити кластеризацію з використанням цього методу та візуалізацію її результатів (Рис.3) за допомогою бібліотеки **scikit-learn**.

```
# Імпорт бібліотек
from sklearn import datasets
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Завантаження датасету
iris_df = datasets.load_iris()

# Визначаємо модель та швидкість навчання
model = TSNE(learning_rate=100)

# Навчаємо модель
transformed = model.fit_transform(iris_df.data)

# Представляємо результат у двовимірних координатах
x_axis = transformed[:, 0]
y_axis = transformed[:, 1]

plt.scatter(x_axis, y_axis, c=iris_df.target)
plt.show()
```

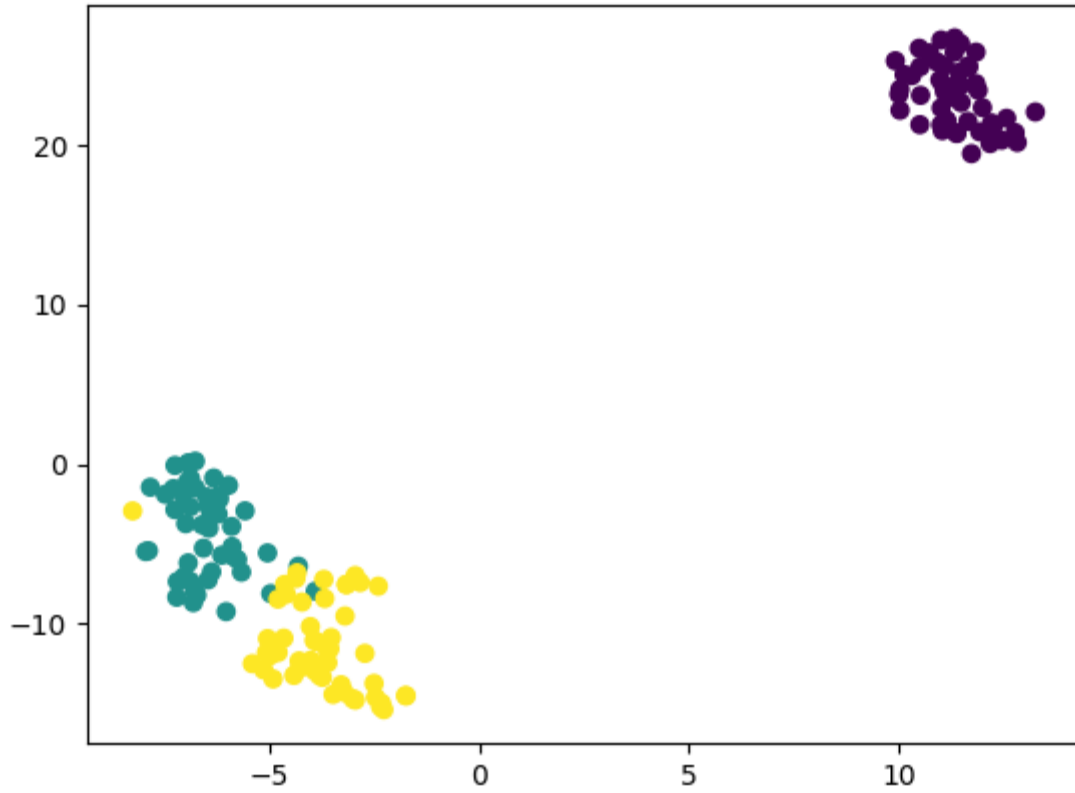


Рис.3. Візуалізація результатів кластеризації за методом t-SNE

У датасеті кожен екземпляр представлений чотирма координатами. Таким чином, при відображенні ознак на площину розмірність простору знижується з чотирьох до двох.

Завдання 4: Представте результат у тривимірному просторі. Порівняйте результати кластеризації датасету "Іриси Фішера" методами k-середніх та t-SNE, зробіть висновки.

5. Метод кластеризації на основі щільності DBSCAN

Метод DBSCAN (англ. Density-Based Spatial Clustering of Applications with Noise - алгоритм на основі щільності просторової кластеризації з присутністю шуму) є

популярним алгоритмом кластеризації, який використовується при аналізі даних як одна з альтернатив методу k-середніх.

Метод не вимагає попередніх припущень про кількість кластерів, але потрібно налаштувати два інші параметри: **eps** та **min_samples**. Дані параметри – це відповідно максимальна відстань між сусідніми точками та мінімальна кількість точок на околиці (кількість сусідів), коли можна говорити про те, що ці екземпляри даних утворюють один кластер. У бібліотеці **scikit-learn** є відповідні значення параметрів за замовчуванням (у розглянутому коді ці параметри взяті за замовчуванням), але, як правило, їх доводиться налаштовувати самостійно.

```
# Імпортуємо бібліотеки
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA

# Завантажуємо датасет
iris = load_iris()

# Визначаємо модель
dbscan = DBSCAN()

# Навчаємо
dbscan.fit(iris.data)

# Зменшуємо розмірність за допомогою метода PCA - головних компонент
pca = PCA(n_components=2).fit(iris.data)
pca_2d = pca.transform(iris.data)

# Будуємо у відповідності до трьох класів
for i in range(0, pca_2d.shape[0]):
    if dbscan.labels_[i] == 0:
        c1 = plt.scatter(pca_2d[i, 0], pca_2d[i, 1], c='r', marker='+')
    elif dbscan.labels_[i] == 1:
        c2 = plt.scatter(pca_2d[i, 0], pca_2d[i, 1], c='g', marker='o')
    elif dbscan.labels_[i] == -1:
        c3 = plt.scatter(pca_2d[i, 0], pca_2d[i, 1], c='b', marker='*')

plt.legend([c1, c2, c3], ['Кластер 1', 'Кластер 2', 'Шум'])
plt.title('DBSCAN знайшов 2 кластери та шум')
plt.show()
```

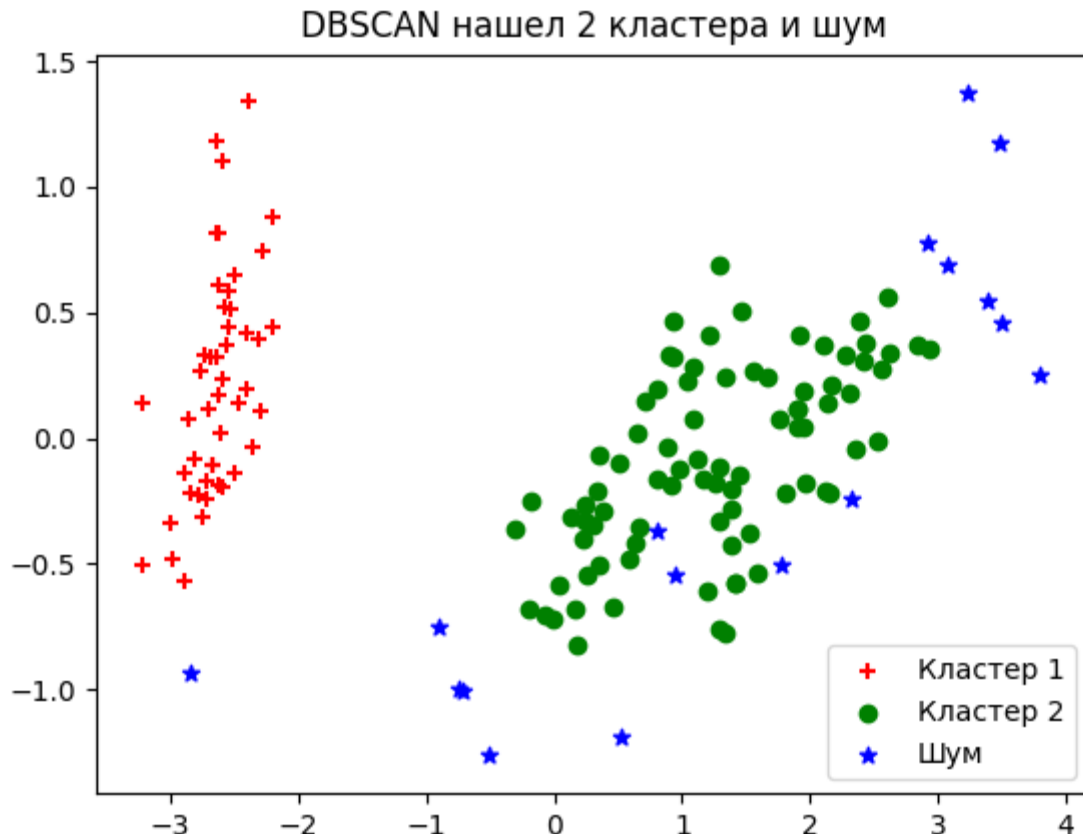


Рис.4. Візуалізація результатів кластеризації методом DBSCAN

Докладніше, у простому викладі даний метод розглянуто [у статті на Habr](#).

Завдання 5: Вивчіть докладніше цей метод і метод головних компонентів PCA (<https://habr.com/ru/post/304214/>), змініть параметри **eps** та **min_samples**, досягніть більш точної кластеризації розглянутого датасета. Зробіть висновки про роботу цього методу у порівнянні з методами, розглянутими раніше.

6. Зміст звіту:

- коди написаних (змінених) алгоритмів із коментарями відповідно до завдань;
- програми, що реалізують розглянуті методи відповідно до завдань;
- візуалізація результатів роботи методів відповідно до завдань;

- висновки щодо кожного розділу лабораторної роботи.